SCALER School of Technology · Priyansh Saxena

ML Systems 101: From Jupyter to Production

Production Gap ▪ System Anatomy ▪ Technical Debt ▪ Design Framework

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 1 — Session 1/2

ROADMAP

Agenda

1. The Notebook-to-Production Gap
2. Anatomy of a Production ML System
3. Technical Debt in ML Systems
4. The ML System Design Framework

TOC

1

The Notebook-to-Production Gap

1 You trained a model. Now what?

What you built

- A Jupyter notebook
- Accuracy: 94% on the test set
- Runs on your local GPU
- `model.pkl` sitting in `/tmp`
- You: "It works!"

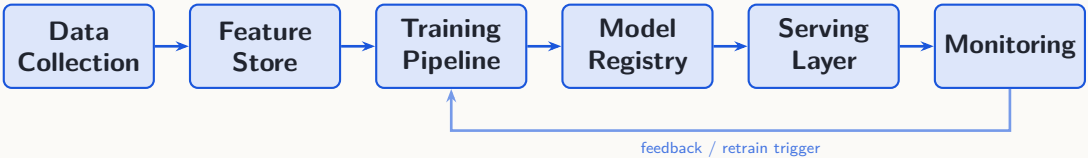
What production needs

- 24/7 availability (99.9% uptime)
- Latency SLA $\leq$ 100 ms p99
- Input validation & error handling
- Monitoring and alerting
- Automated retrain pipeline
- Rollback strategy

The gap

Everything between a working notebook and a reliable production system is **ML Systems Engineering**.

1 The six components of a production ML system



- Each component is independently deployable, versioned, and monitored
- Failure in *any* component can silently degrade model performance
- This course covers every box in depth over 12 weeks

1 Failure Mode 1 — Training-Serving Skew

What happens

- You fit a `StandardScaler` on training data
- You *forget* to save it alongside the model
- At serving time, raw (unscaled) features go in
- Predictions are garbage — silently

Root cause

- Feature preprocessing is **not** part of the saved artefact
- Training and serving code diverge over time

The fix

- Save the *entire* preprocessing pipeline, not just model weights
- Use `sklearn.pipeline.Pipeline` or equivalent
- Validate feature statistics at serving time
- Assert schema match: training $\equiv$ serving

1 Failure Mode 2 — Silent Degradation & Data Drift

What happens

- Model trained Jan 2020, deployed Feb 2020
- No monitoring, no retraining schedule
- The world changes: user behaviour, market, language
- Accuracy quietly falls from 94% to 61%
- Nobody notices for **6 months**

Types of drift

- **Data drift** — input distribution $P(X)$ shifts
- **Concept drift** — $P(Y|X)$ shifts
- **Label shift** — output distribution $P(Y)$ shifts

Real example

Travel demand models trained pre-COVID, deployed 2021. $P(\text{travel}|\text{flight search})$ collapsed overnight. No monitoring $\Rightarrow$ bad recs for months.

The fix

- Monitor input feature distributions daily
- Track prediction distribution vs. baseline
- Set up automated retrain triggers

1 Failure Mode 3 — Feedback Loops & Technical Debt

Feedback loops

- Your model's predictions *become* future training data
- Recommender shows only pop music ⇒ only pop music clicked ⇒ model learns: show more pop
- Self-reinforcing bias, no external correction signal
- Called **exposure bias** or **closed-loop amplification**

Technical debt (Sculley et al. 2015)

- **Glue code** — 500 lines of hacky wrappers to connect components
- **Pipeline jungles** — no one knows what feeds what
- **Dead experimental codepaths** — "we might need this later"
- **Undeclared consumers** — models reading each other's outputs silently

Key insight

ML debt compounds faster than software debt because **data dependencies are invisible**.

1 Quiz 1

Q1 — Amazon Hiring Algorithm

Amazon built a resume screening model trained on historical hiring data. It systematically downranked resumes containing the word "women's" (e.g. women's chess club).

Which failure mode does this illustrate? Explain in one sentence.

Q2 — Travel Prediction

A travel demand model was trained on 2018–2019 data and deployed in March 2021. Predictions were wildly off for months before anyone noticed.

Which failure mode does this illustrate? What should have caught it earlier?

1 Quiz 1 — Answers

A1 — Amazon Hiring Algorithm

Failure mode: Feedback loop / training data bias.

The model was trained on historical hiring decisions made by humans who systematically favoured male candidates. The model learned and amplified that bias — a classic case of **closed-loop amplification** where biased labels become biased predictions.

A2 — Travel Prediction

Failure mode: Concept drift (silent degradation).

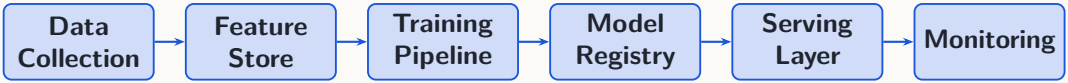
COVID-19 caused $P(Y|X)$ — the relationship between search signals and actual travel — to collapse overnight. What should have caught it earlier: **monitoring the prediction distribution** (output drift) and **business metrics** (actual booking rates). A daily drift alert on feature distributions would have fired within days of lockdowns starting.

2

Anatomy of a Production ML System

2 Anatomy of a Production ML System

We will walk through each of the six components.



- Slides 10–15 cover each component: purpose, key concerns, and tooling
- Real systems rarely look this clean — but understanding each piece lets you reason about the mess

2 Data Collection

Ingestion patterns

- **Event streams** — Kafka, Kinesis; real-time user actions
- **Batch ingestion** — nightly ETL from data warehouse
- **Third-party APIs** — weather, financial feeds
- **Human labels** — crowd-sourced annotation pipelines

Key decisions

- How often to collect? (frequency vs. cost)
- Where to store raw vs. processed?
- Who owns schema evolution?

Data quality checks (non-negotiable)

- **Schema validation** — expected columns, dtypes
- **Null/missing rate** — per-column thresholds
- **Range checks** — $age \in [0, 130]$
- **Referential integrity** — foreign keys exist
- **Volume checks** — sudden drop $\Rightarrow$ pipeline failure

Tools: Great Expectations, dbt tests, Soda Core

2 Feature Store

Offline store (training)

- Historical feature values for training
- Stored in column-oriented format (Parquet, Delta Lake)
- **Point-in-time correctness**: value at t uses only data available *before* t
- Prevents **data leakage**

Online store (serving)

- Low-latency lookup at inference time
- Stored in key-value store (Redis, DynamoDB)
- Must stay in sync with offline store
- Staleness budget: how old is too old?

Why not just recompute features at serving time?

Cost, latency, and reproducibility. A feature store lets you **compute once, use everywhere**. Training and serving use *identical* feature logic.

Tools: Feast (open source), Tecton (managed), Hopsworks

2 Training Pipeline

What the pipeline does

- Pulls versioned data from feature store
- Runs preprocessing, feature engineering
- Trains model with tracked hyperparameters
- Evaluates on held-out validation set
- Pushes artefact to model registry if metrics pass
- Triggers retraining on schedule or drift alert

What must be tracked

- **Data version** — which dataset snapshot?
- **Code version** — git commit hash
- **Hyperparameters** — every knob
- **Metrics** — train, val, test; not just accuracy
- **Artefacts** — model file, preprocessor, config

Tools: MLflow, Weights & Biases, DVC, Kubeflow Pipelines

2 Model Registry

More than a file on S3

- Centralised catalogue of trained models
- Each model has: version, lineage, metrics, tags
- Lifecycle states: **Staging** → **Production** → **Archived**
- Promotes collaboration: data scientist trains, platform team deploys
- Enables **instant rollback**: demote bad model, promote previous

Registry vs. S3 bucket

S3 bucket	Model registry
File storage	Metadata + lineage
No versioning	Semantic versions
No lifecycle	Staging/Prod/Archive
Manual discovery	Search & filter
No audit trail	Full audit log

Tools: MLflow Registry, W&B Registry, Vertex AI Model Registry

2 Serving Layer

Serving patterns

- **Synchronous REST/gRPC** — client waits; use when latency $< 1\,\text{s}$ required
- **Async batch** — process queued jobs; use for offline scoring
- **Streaming** — continuous inference on event stream; use for real-time pipelines
- **Edge** — model runs on device; use for privacy / offline

Latency budgets

- Define SLA *before* choosing model size
- Measure p50, p95, **p99** — not mean
- Budget = feature lookup + preprocessing + inference + postprocessing
- Large model + no caching + slow feature lookup = SLA breach

Tools: BentoML, Triton Inference Server, TorchServe, Ray Serve

2 Monitoring

What to monitor

- **Input drift** — are features shifting from training distribution?
- **Output drift** — is prediction distribution changing?
- **Data quality** — nulls, schema violations, volume drops
- **Business metrics** — click rate, conversion, revenue impact
- **System metrics** — latency, error rate, throughput

Alerting hierarchy

1. **Data quality alert** — pipeline may be broken
2. **Input drift alert** — consider retraining
3. **Output drift alert** — investigate urgently
4. **Business metric drop** — model or product issue

Tools: Evidently AI, Arize, WhyLabs, Prometheus + Grafana

2 Quiz 2

Q1 — Offline vs. Online Feature Store

You are building a fraud detection model. During training you compute a feature: *"number of transactions by this user in the past 7 days"*.

Should this feature come from the offline store or the online store at serving time? Why?

Bonus: What goes wrong if you compute it fresh from the database every request?

Q2 — Registry vs. S3

Your team stores trained model files in an S3 bucket with names like `model_v3_final_FINAL.pkl`.

Name two concrete problems this causes in a team of 5 data scientists, and how a model registry fixes them.

2 Quiz 2 — Answers

A1 — Offline vs. Online Feature Store

Online store — at serving time you need the feature instantly (low latency), so it must be pre-computed and cached in a key-value store (e.g. Redis).

Bonus: Computing fresh from the DB every request adds **50–200 ms** of query latency, risks **SLA breach**, and creates a **training-serving skew** if the DB query logic differs from how it was computed at training time.

A2 — Registry vs. S3

Problem 1 — No reproducibility: `_final_FINAL.pkl` has no lineage. You cannot tell which dataset or code version produced it, so debugging a production regression is impossible.

Problem 2 — No rollback: Without versioned lifecycle states (Staging → Production → Archived), reverting a bad model means manually hunting for the previous file.

Registry fix: Semantic versions + lineage metadata + one-click promotion/demotion solves both.

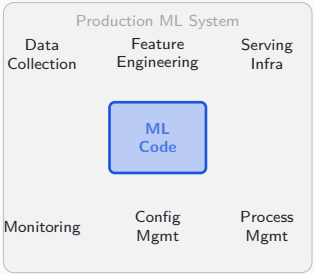
3

Technical Debt in ML Systems

3 Hidden Technical Debt in ML Systems

Sculley et al., Google (NeurIPS 2015)

- Seminal paper from Google's production ML team
- Key finding: **ML code is a small fraction of a production ML system**
- The majority of code is infrastructure, pipelines, and glue
- All of it accrues debt — but ML debt is *harder to see*
- Data dependencies are "**entanglement**" — changing one input changes everything



Adapted from Sculley et al. 2015

3 Pipeline Jungles

How it starts

- Team A writes a pipeline for feature f_1
- Team B copies and modifies it
- Now 3 versions exist; nobody knows which feeds prod
- "Undeclared consumers" read outputs silently

Signs of a pipeline jungle

- No one can draw the data flow end-to-end
- Deleting an old table breaks something unexpected
- Reprocessing old data gives different results

Real cost

At one large company, 30% of ML engineer time was spent **understanding existing pipelines** rather than building new ones. (Sculley et al.)

The fix

- Explicit data lineage tracking
- Single source of truth per feature (feature store)
- Ownership tags on every pipeline
- Regular audits — delete dead pipelines

3 Glue Code & Correction Cascades

Glue code

- Generic open-source model wrapped in 1000 lines of custom glue
- Glue handles: edge cases, format conversion, schema mismatches, post-processing hacks
- Makes swapping the model **nearly impossible**
- The glue, not the model, carries the real logic

Correction cascades

- Model A is slightly wrong for edge case e
- Team adds Model B on top to fix e
- Model B causes error on edge case f
- Team adds Model C ...
- Now you have a cascade: $A \rightarrow B \rightarrow C \rightarrow \ldots$
- Changing A breaks B, C, ...

The danger of both

Neither glue code nor correction cascades show up in model accuracy metrics. They only surface when you try to **change something**. The cost is **velocity loss**: every new feature takes 3× longer.

3 The Debt Meter

Debt pattern	Where it comes from	How to fix it
Training-serving skew	Preprocessing not saved with model; serving code diverges from training code	Use `Pipeline` objects; validate feature stats at serving time
Pipeline jungle	Organic growth, no ownership, undeclared consumers	Data lineage tool; feature store; ownership tagging
Glue code	Wrapping general-purpose libraries without abstraction	Refactor to clean API; replace glue with proper integration layer
Correction cascades	Patching model outputs instead of fixing root cause	Fix upstream model or data; avoid chaining model outputs

5-Minute Break

Grab some water or take a short walk. Think about a model you have built or used.

Which failure mode would bite it first?

4

The ML System Design Framework

4 The 5-Question Framework

1. What problem?	2. What data?	3. What model?	4. How to serve?	5. How to monitor?

- **Q1 — What problem?** Define the ML objective, success metric, and business value
- **Q2 — What data?** Sources, volume, freshness, labelling cost, legal constraints
- **Q3 — What model?** Complexity vs. latency vs. interpretability trade-offs
- **Q4 — How to serve?** Sync vs. async, latency SLA, scaling requirements
- **Q5 — How to monitor?** What breaks first? How do you detect it? Who gets paged?

4 Applying the Framework: Spam Classifier

Question	Answer for spam detection
1. What problem?	Binary classification: is this email spam? Success metric: precision $\geq$ 99% (false positives delete real email), recall $\geq$ 95%
2. What data?	Email text + metadata (sender, subject, headers). Labels: user "mark as spam" actions. 500M emails/day, retention 90 days. GDPR applies.
3. What model?	Start: logistic regression on TF-IDF (fast, interpretable, low latency). Iterate to: fine-tuned BERT if precision insufficient.
4. How to serve?	Synchronous REST, $\leq$ 50 ms SLA (email arrives, classify before inbox delivery). Deploy as sidecar to mail server.
5. How to monitor?	Track: % emails marked spam by users post-delivery (false negatives), % spam complaints on ham (false positives). Retrain weekly.

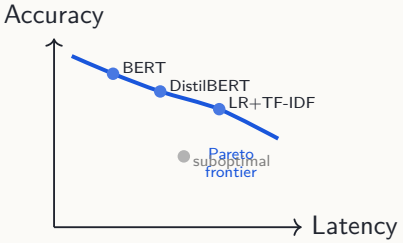
4 Trade-off 1 — Latency vs. Accuracy

The fundamental serving tension

- Larger models ⇒ higher accuracy, higher latency
- Quantisation, distillation, pruning recover accuracy at lower cost
- **Latency SLA** drives model choice, not vice versa
- Caching: cache hit = 0 ms latency

Rules of thumb

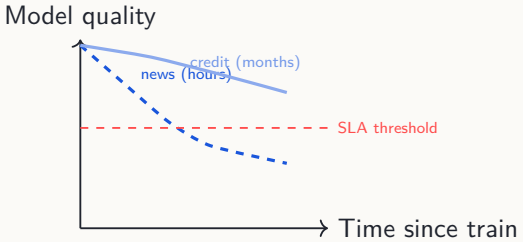
- Mobile/real-time: $<$50 ms ⇒ small model, edge
- API product: $<$300 ms ⇒ medium model, GPU
- Batch overnight: no SLA ⇒ biggest affordable model



4 Trade-off 2 — Freshness vs. Cost

How often to retrain?

- Retraining is *not free*: compute, eng. time, regression risk
- Staleness curve is domain-specific
- News recommendation: degrades in **hours**
- Credit scoring: degrades over **months**
- Medical imaging: stable for **years**



Retraining triggers

- **Scheduled** — weekly/daily regardless of drift
- **Drift-triggered** — stat. test fires when $P(X)$ shifts
- **Performance-triggered** — metric drops below threshold

4 When to Use Which Serving Pattern

Pattern	When to use	Examples
Synchronous REST	User is waiting; latency < 1 s; single item at a time; simple scaling	Spam filter, fraud check at checkout, search ranking
Asynchronous batch	High volume, no real-time need; overnight or hourly runs; cost-sensitive	Monthly churn prediction, weekly recommendation refresh, nightly demand forecast
Streaming (Kafka + Flink)	Continuous event stream; sub-second freshness; stateful aggregations across events	Real-time fraud detection, dynamic pricing, live anomaly detection
Edge / on-device	Privacy constraints; offline use; ultra-low latency ($<$10 ms); no cloud cost	Face unlock, keyword spotting, autonomous vehicle perception

4 What We Covered Today

§1 Notebook-to-Production Gap

- Six components of a production ML system
- Training-serving skew
- Silent degradation & data drift
- Feedback loops & technical debt

§2 Anatomy of a Production ML System

- Data Collection, Feature Store, Training Pipeline
- Model Registry, Serving Layer, Monitoring
- Key concerns & tooling per component

§3 Technical Debt in ML Systems

- Sculley et al. 2015: ML code is a small fraction
- Pipeline jungles
- Glue code & correction cascades

§4 ML System Design Framework

- The 5-question framework
- Applied to spam classifier
- Latency vs. accuracy trade-off
- Freshness vs. cost trade-off
- Serving pattern selection

Thank You!

Shipped to prod at 2am. Monitoring optional. What could go wrong?



Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design